



UNIVERSIDAD ESTATAL A DISTANCIA
VICERRECTORÍA ACADÉMICA
ESCUELA DE CIENCIAS EXACTAS Y NATURALES



Cátedra Tecnología de Sistemas

Proyecto 2. Valor 2%

1. Temas de Estudio

- Tema 5 - Subprocesamiento múltiple
- Tema 6 - Bases de datos y componentes ADO.NET
- Tema 7 - Colecciones
- Tema 8 - Redes

2. Objetivo

Aplicar los conceptos de subprocesamiento múltiple, bases de datos, colecciones y redes en la evolución del juego de batallas desarrollado en el Proyecto #1, asimismo, reforzar el uso de interfaz gráfica y el manejo de excepciones, integrando todos estos elementos en una arquitectura cliente/servidor más robusto.

3. Software de Desarrollo

Visual Studio Community 2022 (C# Net 8.0)

4. Desarrollo

Los encargados de la empresa MITICAX se encuentran satisfechos con la aplicación previamente desarrollada, sin embargo, solicitan realizar ciertos ajustes para mejorar su rendimiento y ampliar sus funcionalidades. Estas mejoras incluyen:

- Incorporar el almacenamiento de los datos en una base de datos SQL Server para conservar la información de forma persistente.

- Agregar una funcionalidad de interacción en red, implementando una arquitectura cliente/servidor, que permita a los jugadores conectarse.

El sistema por desarrollar debe componerse, como mínimo, **de dos soluciones independientes**; una aplicación cliente y una aplicación servidor. Ambas deben comunicarse a través de la red utilizando el protocolo TCP.

- La aplicación cliente será utilizada por los jugadores para iniciar sesión, adquirir criaturas, gestionar el inventario, dar mantenimiento a sus equipos y participar en batallas.
- La aplicación servidor será operado por un único administrador en las oficinas centrales. Su función será mantener actualizadas las criaturas disponibles para la venta y para agregar a los jugadores.

NOTA: Toda la información manejada por el sistema debe almacenarse en una base de datos SQL Server. **Es importante recalcar que la aplicación cliente nunca tendrá acceso directo a la base de datos, la única forma de enviar y recibir información será únicamente conectándose al servidor mediante el protocolo TCP.**

SERVIDOR

1. La aplicación servidor debe cumplir con los siguientes criterios:

- Implementar un servidor capaz de escuchar y responder solicitudes de red de múltiples clientes mediante el protocolo TCP, utilizando subprocesamiento múltiple para atender conexiones simultaneas.
- Utilizar sockets en la dirección 127.0.0.1 y el puerto 14100 para recibir solicitudes desde la aplicación cliente.
- Almacenar toda la información en una base de datos SQL Server (Es importante indicar que la comunicación con la base de datos debe realizarse mediante ADO.NET).
- Recibir desde la aplicación cliente la información generada por los jugadores, procesarla y registrarla en la base de datos. Todas las consultas o actualizaciones que soliciten los clientes deben realizarse a través del servidor.
- **Se recuerda que solo la aplicación servidor se comunica con la base de datos, la aplicación cliente no tiene conexión directa con la base de datos.**
- Ser utilizada por un único usuario administrador en las oficinas centrales, el cual no requiere registrarse, tampoco requiere usuario y contraseña.
- Incluir una pantalla principal tipo bitácora que muestre en tiempo real **todos los eventos** enviados por todos los clientes conectados, como, por ejemplo: conexiones y desconexiones al servidor, Batallas activas u otros eventos relevantes.
- Mostrar en todo momento la cantidad de clientes conectados.

- El servidor debe soportar un máximo de 8 conexiones simultáneas, una novena conexión deberá esperar hasta que alguna de las 8 conexiones establecidas se desconecte.
- El servidor debe contar con un menú principal con las siguientes opciones:
 - Registrar y consultar Criaturas.
 - Registrar y consultar Jugadores.

2. Registrar y consultar Jugadores

Esta opción permitirá al administrador registrar o consultar la información de los jugadores. Para ello se debe utilizar una tabla denominada “Jugadores” en la base de datos.

Los datos que deben registrarse son los siguientes:

- IdJugador (int), identificador único del jugador. Este valor debe ser único por lo que se debe validar en el sistema que no existan valores repetidos.
- Nombre (string), nombre del jugador.
- Usuario (string), Apodo o “Nick” que el jugador usará para iniciar sesión. Este valor debe ser único en la base de datos
- Password (string), contraseña utilizada para ingresar al sistema.
- FechaNacimiento (datetime), fecha de nacimiento del jugador. Se debe validar que el jugador tenga más de 10 años.
- Nivel (int), Determina el nivel de experiencia del jugador. En el momento de registrarlo el nivel es 01-Novato, si llega a ganar 5 batallas sube a nivel estudiante, si llega a ganar 10 batallas sube de nivel a maestro y si llega a ganar 20 batallas sube a nivel supremo. El nivel siempre debe ser un valor válido, iniciando en 1.
- Cristales (int), moneda utilizada en el juego, al ser registrado todo jugador tendrá 100 Cristales. Este valor no puede ser negativo.
- Todos los campos son obligatorios.
- Al agregar un nuevo jugador, la información debe reflejarse de automáticamente en el DataGridView correspondiente.

3. Registrar y consultar Criaturas

Esta opción permitirá al administrador registrar y consultar la información de las distintas criaturas disponibles para las batallas. Esta lista representará las posibles criaturas que los jugadores podrán adquirir y utilizar en el juego.

Para esta funcionalidad se debe utilizar una tabla denominada “TiendaCriaturas” en la base de datos, donde se almacenará toda la información registrada.

Los datos que deben registrarse son los siguientes:

- IdCriatura (int), identificador único de la criatura. Este valor debe ser único por lo que se debe validar en el sistema que no existan valores repetidos.
- Nombre (string), nombre de la criatura.
- Tipo (string), solo admite los valores: agua, tierra, aire o fuego.
- Nivel (int), Nivel de criatura. Los niveles son:
 - o 01-Iniciado
 - o 02-Aprendiz
 - o 03-Estudiente
 - o 04-Avanzado
 - o 05-Maestro
- Poder (int), Determina el nivel de fuerza de la criatura. Debe ser mayor o igual a cero.
- Resistencia (int), determina el nivel de resistencia ante los ataques. Debe ser mayor o igual a cero.
- Costo (int), Este es el valor en la tienda de la criatura. La moneda se llama cristales, así que, si el valor se define en 10, son 10 cristales. El costo debe ser mayor o igual a cero y cumplir el siguiente rango:

NIVEL CRIATURA	Rango de Costo
Iniciado	< 100
Aprendiz	>=100 y < 300
Estudiante	>=300 y < 600
Avanzado	>=600 y < 1200
Maestro	>=1200

- Todos los campos son obligatorios.
- La aplicación debe mostrar un control “DataGridView” que despliegue la información completa de todas las criaturas registradas.

- Cada vez que se agregue una nueva criatura, deberá verse reflejada de manera automática en el DataGridView.

CLIENTE

La aplicación cliente debe cumplir con los siguientes requisitos:

Cuando en el Cliente se indica que se debe guardar información en la base de datos (Inventario, Equipos, Batallas o Rondas), los datos corresponden a los mismos campos definidos en el Proyecto I para cada entidad. Aunque no se listan de forma individual en esta sección, deben considerarse idénticos a los ya establecidos, y serán almacenados de manera persistente en la base de datos a través del Servidor.

1. Conexión y Validación

- Permitir que el jugador se conecte y se desconecte del servidor central utilizando sockets en la dirección 127.0.0.1 y el puerto 14100 para enviar y recibir solicitudes.
- Se requiere una opción para iniciar sesión mediante usuario y contraseña.
- Validar que el jugador exista en la base de datos **antes de mostrar las funcionalidades** de la aplicación cliente.
- El jugador debió ser registrado previamente por el administrador desde la aplicación servidor.
- Una vez validado, la aplicación cliente debe mostrar el nombre del jugador, esta información debe ser siempre visible en la aplicación cliente. Además, una vez que el jugador ha sido validado, se muestran desbloqueadas las funcionalidades de la aplicación.

2. Registrar y consultar Inventario de Criaturas de Jugador

Esta opción le permitirá al jugador adquirir nuevas criaturas para su inventario.

- La lista de criaturas disponibles debe obtenerse desde el servidor.
- Un jugador no puede poseer dos criaturas del mismo tipo, por lo tanto, al mostrar las criaturas disponibles solo deben visualizarse aquellas del tipo que aún no posea.
- No se permite registrar dos veces la misma criatura (IdCriatura) en el inventario de un mismo jugador.
- Al seleccionar una criatura el sistema debe:
 - Validar si la cantidad de Cristales que posee el jugador es suficiente para “pagar” por la criatura,
 - Si cumple con el requisito, se permite agregar la criatura al inventario personal del jugador y se descuentan los Cristales correspondientes.
 - En caso de no contar con los Cristales suficientes se debe mostrar un mensaje que indique “No posee la cantidad de cristales suficientes para obtener la criatura”.
 - Todos los datos del inventario de criaturas deben ser almacenadas en la tabla InventarioJugador de la base de datos, de tal forma que cada vez que el usuario ingresa pueda visualizar completo su inventario de criaturas.

- Se deberá mostrar un control “DataGridView” que muestre todo el inventario de criaturas del usuario. Cuando se agrega una nueva criatura al inventario deberá verse reflejada de manera automática en el DataGridView, es decir deberá actualizarse.
- En InventarioJugador, los atributos Poder y Resistencia representan valores dinámicos que pueden variar según las batallas, mientras que en TiendaCriaturas se mantienen los valores base.
- Además, a partir de este inventario el jugador podrá crear y administrar equipos de tres criaturas, los cuales quedarán registrados para ser utilizados en las batallas.
 - Los equipos deben estar conformados por tres criaturas distintas y todas ellas deben pertenecer previamente al inventario del jugador.
 - Cada criatura seleccionada en el equipo debe pertenecer previamente al inventario del jugador.

3. Registrar o iniciar Batalla

Esta opción al igual que en la aplicación original, permitirá a dos jugadores participar en una batalla. Sin embargo, el funcionamiento tendrá algunos cambios:

- En esta pantalla el jugador podrá escoger alguno de los equipos que haya creado previamente. Dicho equipo debe estar conformado por tres criaturas distintas que pertenezcan al inventario del jugador.
- Una vez que el jugador tenga seleccionado el equipo, deberá presionar el botón “iniciar batalla”. Al hacerlo, la aplicación realizará una verificación en el servidor para determinar si existe otro jugador disponible que ya haya iniciado una solicitud de batalla:
 - Si no hay un jugador disponible, entonces se iniciará una solicitud de batalla y la aplicación quedará en espera hasta que otro jugador se conecte a la batalla (solo pueden participar dos jugadores por batalla).
 - Si sí hay un jugador disponible, el sistema iniciará la batalla de forma inmediata. En este caso se guardarán los datos de la batalla en la tabla Batalla, se generará un código único de batalla y se registrarán los datos de los equipos seleccionados por cada jugador, junto con la fecha de la batalla.
 - Posteriormente se iniciará la ejecución de la batalla y el almacenamiento de las rondas. Es importante que durante todo el proceso las rondas se reflejen en la pantalla de cada usuario, indicando quién gana cada ronda y quién es el ganador de la batalla. Cada ronda se debe registrar en la tabla Rondas, con los siguientes campos: IdRonda, IdBatalla, IdInventarioAtacante, IdInventarioDefensor y GanadorRonda.
 - Reglas adicionales para asegurar la integridad de las rondas:
 - Para una misma batalla, un jugador no puede utilizar la misma criatura en más de una ronda.
 - El par (IdBatalla, IdRonda) debe ser único en la base de datos, asegurándose que cada ronda quede registrada una sola vez.

- El campo IdRonda solo admite los valores 1,2 o 3.
- El campo GanadorRonda puede permanecer vacío mientras la batalla está en curso, al registrarse debe corresponder al id de uno de los dos jugadores participantes.
- Cuando un jugador gane una batalla, se debe actualizar su registro en la base de datos (tabla Batallas), indicando quien fue el ganador de la batalla.
- La aplicación debe mostrar un control “DataGridView” con el historial de batallas en las que ha participado el jugador. Cada vez que se registre una nueva batalla, esta deberá reflejarse de manera automática en el DataGridView.
- Al finalizar la batalla, el sistema debe presentar en una pantalla un resumen que indique las recompensas obtenidas por cada jugador, como la cantidad de cristales y/o el aumento de poder de las criaturas.
- Las reglas para determinar al ganador de cada ronda y de la batalla son las mismas establecidas en el Proyecto 1.

4. Consultas

La aplicación cliente debe incluir opciones para realizar las siguientes consultas:

- Consultar el top 10 de ganadores, mostrando los diez jugadores con mayor número de batallas ganadas. La cantidad de batallas ganadas se debe calcular de acuerdo con la información almacenada en la tabla de base de datos Batallas.
- Listar todas las batallas en las que ha participado un jugador específico
- Presentar el detalle de las rondas de una batalla específica, indicando cuáles fueron ganadas por cada jugador
- Consultar las criaturas que posee un jugador (inventarios de criaturas)

Se reitera: la aplicación cliente no tiene conexión directa con la base de datos. Todas las solicitudes deben enviarse al servidor, el cual es el único que puede acceder y consultar la base de datos.

5. Consideraciones técnicas

- Uso de comentarios
 - Es obligatorio incluir comentarios en el código durante el desarrollo del programa.
 - En la parte superior de cada archivo cs (clase, program, formulario, etc) se debe especificar la siguiente información:
 - Nombre de la universidad
 - Cuatrimestre
 - Nombre y la descripción del proyecto,
 - Nombre completo del estudiante
 - Fecha en que se desarrolla la programación.

- Este encabezado debe colocarse en todos los archivos del proyecto y servir como referencia para identificar el código, como se puede observar en el siguiente ejemplo:

```
1  /**
2   * UNED X Cuatrimestre
3   * Proyecto X: Descripción del proyecto
4   * Estudiante:
5   * Fecha:
6   */
7  namespace CapaEntidades
8  {
```

- Debe utilizar POO (Programación orientada a objetos) para resolver el problema
 - Las clases de objetos que usted considere como entidades no deben contener lógica para solicitar información a la persona usuaria, únicamente deben incluir la estructura de la clase y sus propiedades.
 - Los nombres de las clases del objeto y de las propiedades no deben de llevar tilde ni la letra ñ para evitar problemas de compatibilidad.
 - En la solución, se deben crear proyectos independientes para implementar las distintas capas recomendadas:
 - Capa de Entidades,
 - Capa de Acceso a Datos,
 - Capa de Lógica Negocio,
 - Capa de Presentación (Interfaz del Usuario), etc.
 - **Estas capas NO deben ser carpetas** dentro de un solo proyecto, sino separados dentro de la misma solución.
- **La única capa con acceso directo a base de datos es la capa de Acceso a Datos. Ninguna otra capa puede consultar o insertar datos en la base de datos.**
- Cada clase debe ser creada en archivo por separado.
- Las clases no deben contener métodos vacíos y es importante utilizar nombres claros y significativos para las variables.
- Uso de combobox
 - Los controles Combobox no deben permitir editar o escribir en él, solo debe permitir seleccionar una opción de la lista.
- Uso de DataGridView
 - Los títulos de las columnas del DataGridView deben configurarse previamente para que se para que se muestren correctamente, y no se debe dejar que tomen el nombre de la propiedad de la clase.
 - Los DataGridView debe de ser de solo lectura, no se pueden modificar los datos que se muestren allí. Tampoco debe permitir agregar o borrar nuevos datos.
 - Cuando se seleccione un dato del DataGridView se debe seleccionar toda la fila.
- Uso de colecciones

- Se deben usar colecciones genéricas (List<>) cuando se requiera consultar de base de datos y retornar más de un registro.
- Interfaz de usuario
 - Debe usar interfaz de usuario con formularios GUI (Proyecto Windows Form).
 - Se debe cuidar con el orden de los formularios.
 - Se debe cuidar con la ortografía.
 - Se debe crear un menú para acceder a las opciones solicitadas.
 - Cada vez que se agrega un nuevo objeto, se deben limpiar los campos del formulario y mostrar el siguiente mensaje: “El registro se ha ingresado correctamente”. Esto con el fin de que el usuario pueda ingresar otro registro.
 - Si en cualquier pantalla de inclusión surge algún error de validación, se debe enviar un mensaje indicando el error específico y no se deben eliminar los datos de pantalla para dar oportunidad al usuario que corrija el error.
- Base de datos
 - La aplicación debe almacenar y consultar la información en una base de datos SQL Server. No se permite otro motor de base de datos.
 - Para la cadena de conexión, se debe utilizar seguridad integrada de Windows y un archivo de configuración “**App.config**”, de preferencia direcciona en este archivo el servidor a localhost.
 - **El script de creación de la base de datos se publicará en la plataforma. No debe modificar el script de base de datos.**
 - La base de datos debe implementar claves primarias y claves foráneas, además de restricciones de unicidad y de dominio, con el fin de garantizar la integridad de datos en todas las entidades, se recomienda crear índices en las columnas que funcionan como claves foráneas.
- Debe implementar el manejo de excepciones
 - Si ocurre una excepción, el sistema no debe cerrarse. Debe mostrar un mensaje claro al usuario e implementar un manejo adecuado de la excepción para evitar pérdida de información o interrupción del flujo de trabajo.
- Otras consideraciones
 - La solución será probada utilizando al menos 5 aplicaciones cliente ejecutándose de manera simultánea junto con una única instancia de la aplicación servidor.
 - Se recomienda crear un proyecto independiente de tipo biblioteca de clases para las entidades que serán utilizadas tanto solución cliente como en la solución servidor. Esto evita duplicar las clases de entidades en la solución cliente y servidor, en estos solo se agrega la referencia del proyecto o del archivo dll.
 - Es importante revisar constantemente los foros de consulta y el foro de interés en la plataforma AprendeU, ahí se aclaran dudas del proyecto o se realizan anuncios que le pueden ser de mucha utilidad en el desarrollo del proyecto.
 - Se les recuerda las tutorías que se realizan oportunamente y que quedan grabadas. En estas se pueden aclarar dudas en el proceso de desarrollo.

- Los proyectos en donde se identifique plagio o una copia textual o similar, tanto del material digital, como de código fuente, serán anulados y se aplicará el reglamento para dichos casos. Si usa código fuente que no es de su autoría debe indicarlo, aunque sea código del tutor.
- Junto con el proyecto debe entregarse un manual breve que describa los pasos necesarios para probar la solución entregada. Debe contener:
 - Imágenes de las pantallas de las soluciones.
 - Portada: nombre del estudiante, curso, cuatrimestre, número de grupo, tutor asignado, nombre Proyecto).
 - Requisitos para la ejecución de la aplicación
 - Pasos detallados para instalar y ejecutar primero la aplicación Servidor y luego la aplicación Cliente.
 - Instrucciones de prueba de las funcionalidades principales: conexión de usuarios, compra de criaturas, creación de equipos, inicio de batalla, registro de rondas y consultas.
 - Capturas de pantalla principales de la solución, tanto del servidor como del cliente.
 - Archivo PDF, nombre NombreApellido-ManuaP2.pdf

Diagrama cliente/servidor

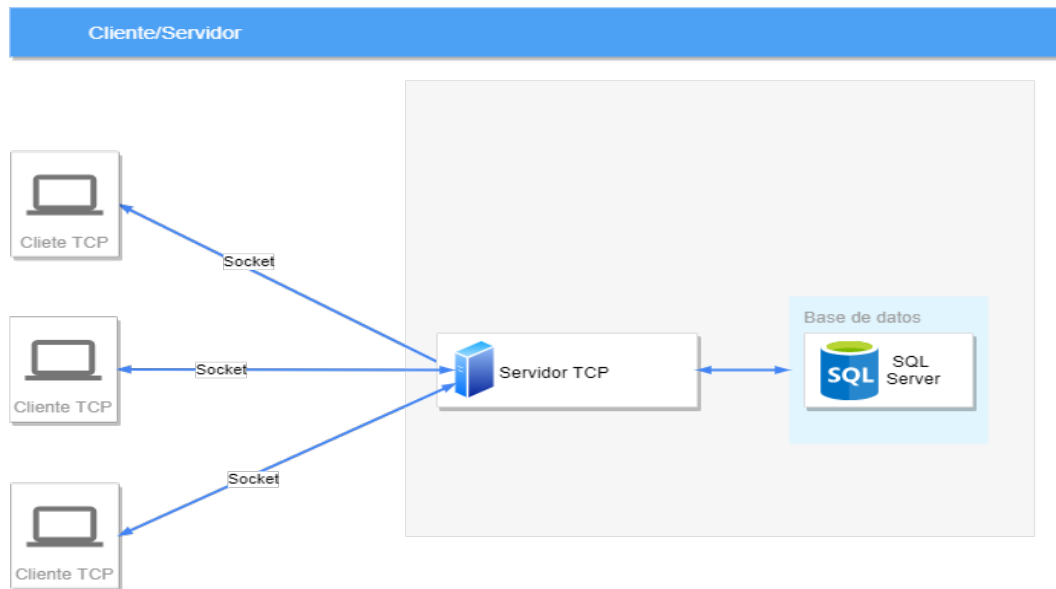
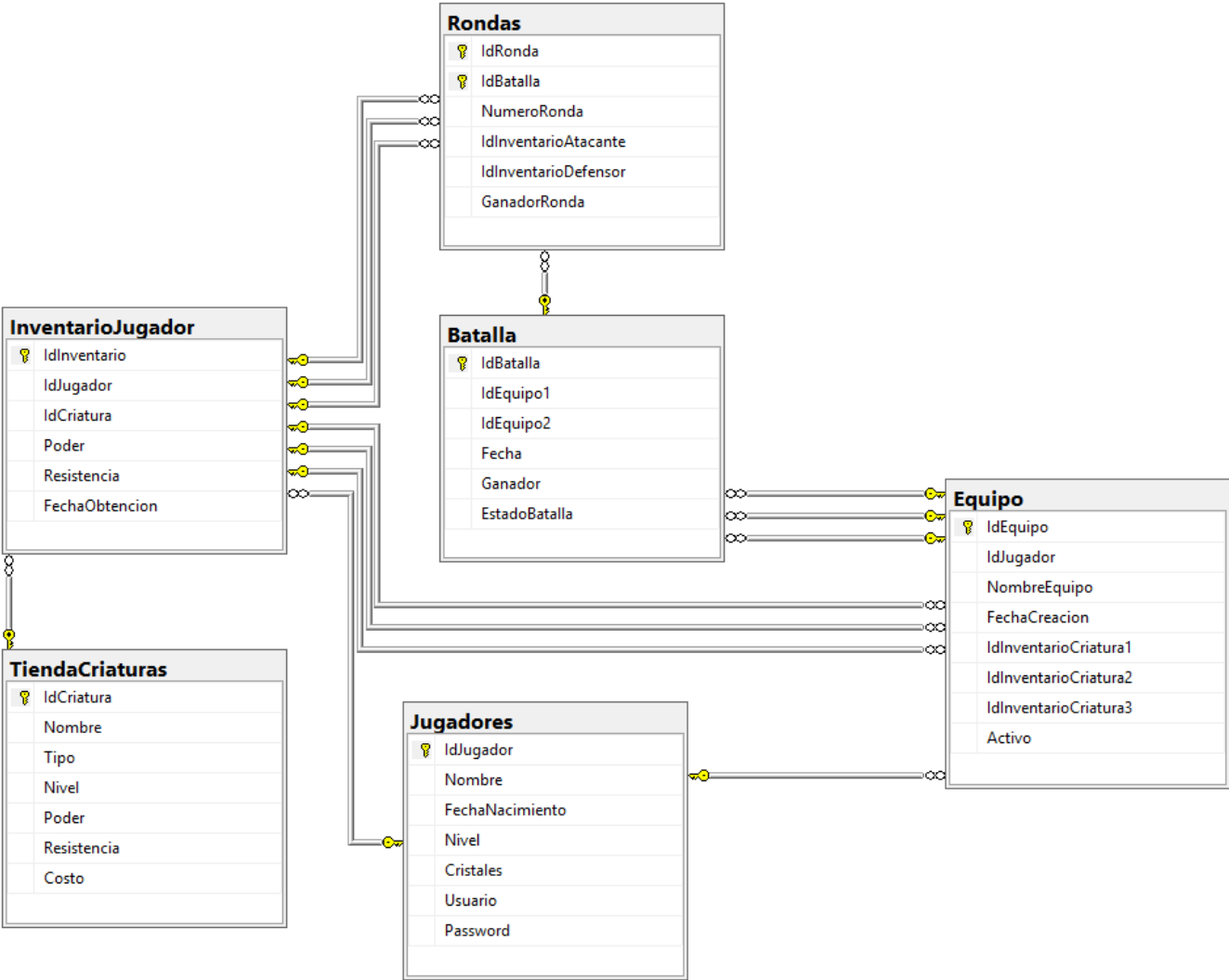


Diagrama base de datos



6. Honestidad Académica



<https://audiovisuales.uned.ac.cr/play/player/23048>

Nota importante

- Cada estudiante es responsable del contenido que entrega.
- Si el archivo enviado no corresponde al proyecto correcto, no se aceptarán entregas posteriores fuera de la fecha establecida.
- Si se detecta que el contenido del archivo coincide con otro estudiante o no es de su autoría, se aplicarán las sanciones indicadas en el documento de Lineamientos ante casos de plagio disponible en la plataforma.

7. Indicaciones Importantes

- Es obligatorio que incluya todo el directorio donde se encuentra el proyecto 2.
- El proyecto 2 debe estar desarrollado en Visual Studio Community 2022 que es la herramienta oficial del curso.
- Los trabajos deben realizarse en forma individual.
- Dentro del código del programa se debe incluir documentación mediante comentarios que expliquen cómo fue desarrollado el proyecto, describiendo el propósito de las clases métodos, y procesos implementados.
- Si utiliza código de algún ejemplo del libro, del algún profesor o de otra fuente que no sea de su autoría, debe de indicarlo.
- Comprima todos los archivos en un solo archivo .zip o .rar.
- **Nombre del archivo que envía:** debe ser nombre y primer apellido del estudiante, y nombre del proyecto. **Ejemplo: JuanRojas-proyecto2.**
- La entrega del proyecto debe ser en las fechas establecidas en la plataforma de aprendizaje en línea AprendeU en el apartado que se indique.
- En caso de concluir el proyecto en el tiempo establecido, el estudiante debe entregar el avance realizado e incluir una carta en formato PDF, en la que explique las razones por las cuales no logró finalizar la totalidad del trabajo.

8. Rúbrica de Evaluación

Criterio	Cumple a satisfacción lo indicado en la evaluación	Cumple medianamente en lo indicado en la evaluación	Cumple en contenido y formato, pero los aportes no son significantes	No cumple o no presenta lo solicitado
Presentación del código: Correcto uso de variables, nombres de métodos y variables significativas, no presenta métodos vacíos. Entrega el archivo del programa con todo el directorio del programa desarrollado. Indica dentro del programa en los comentarios, si se utiliza código de algún ejemplo del libro o algún profesor o de otra fuente que no sea de autoría del estudiante. El nombre del archivo es el solicitado. Agrega los comentarios de información del proyecto en las clases.	5	3	2	0
POO y creación de las clases Utiliza propiedades y no atributos. No agrega constructores ni métodos adicionales. La solución está dividida en capas (proyectos distintos). Solo la capa de acceso a datos tiene acceso a la base de datos.	15	8	4	0
Registra y consulta Criaturas en el servidor: Tipo {agua, tierra, aire, fuego}; Nivel [1..5]; Poder, Resistencia, Costo ≥ 0 ; rango de costo por nivel respetado; DataGridView actualiza al insertar.	5	3	2	0
Registra y consulta Jugadores en el servidor: Usuario único; edad > 10 validada; Cristales iniciales y no negativos; no negativo; DataGridView actualiza al insertar.	5	3	2	0
Registra o se conecta a Batalla, crea rondas y equipos con la información del usuario desde el cliente: Equipo con 3 criaturas	15	8	4	0

distintas del inventario; Rondas: (IdBatalla, IdRonda) único, IdRonda {1,2,3}, un jugador no repite la misma criatura en más de una ronda; Ganador nulo hasta cierre y luego uno de los dos jugadores; historial visible y actualizado.				
Registra y consulta criaturas al inventario del jugador desde el cliente: No duplica IdCriatura para el mismo jugador; verifica y descuenta cristales; DataGridView actualiza al insertar.	5	3	2	0
Consulta top 10 ganadores, batallas, rondas y criaturas, desde el cliente.	7	5	3	0
Conexión de la aplicación cliente con el servidor mediante TCP y administración de las conexiones en el servidor.	10	7	3	0
Validación del jugador en la aplicación cliente.	4	3	2	0
En la aplicación servidor se muestran los eventos enviados por todos los clientes conectados (bitácora).	4	3	2	0
Implementación de un método para administrar la concurrencia en el servidor.	4	3	2	0
Correcto uso del manejo de excepciones con try catch. Mensajes claros ante violaciones de integridad y otras excepciones sin cerrar la aplicación.	3	2	1	0
Interfaz de usuario es fácil de usarse, presenta buen diseño y <i>no presenta faltas de ortografía o redacción</i> . DataGridView en solo lectura, encabezados configurados, selección por fila, refresco automático tras inserciones; combobox sin edición libre; navegación coherente.	5	3	2	0
Integridad en la base de datos y coherencia con reglas del enunciado.	8	5	2	0
Manual de uso de la solución.	5	3	2	0
TOTAL	100			